Algebraic Retrieval: Composable Search for Agents

A Preprint

Damian Delmas
Independent Researcher, Vancouver, BC
damian@getflex.dev

Abstract

An AI agent can compose retrieval by adding and subtracting similarity scores, scaling contributions, and masking candidates before selection. Algebraic Retrieval exposes these operations as mathematical expressions over a SQLite-backed corpus. The agent inspects the available operands, writes a program, and receives a scored relation. An existing materializer executes it. We show three programs alongside full SQL alternatives and native PyTerrier pipelines: signed composition, candidate-pool reranking, and masked weighting. On the public 11,429-document Vaswani fixture, the implementations select the same document sets, with score differences below `1e-6`, but one pair orders differently across scoring paths. The comparisons validate the computation and distinguish score agreement from rank agreement; the interface lets the agent compose it over declared operands.

1 Introduction

Retrieval is often presented to an agent as one search call with a query and a few parameters. For example, an agent may seek implementation architecture and runtime behavior while suppressing marketing material and restricting candidates to recent engineering discussions. These operations can be written together as an expression.

Algebraic Retrieval lets the agent compose that expression at query time. Addition combines compatible similarity scores; subtraction suppresses a direction; multiplication scales a contribution. A mask determines which records participate. The agent can change the program after inspecting its results, using declared symbols instead of reconstructing their underlying joins or Python pipeline bindings. Agreement with established systems checks that the program executes as intended.

The immediate precursor is Programmatic Embedding Modulation (PEM) [5], which exposed query vectors and score arrays inside a SQL/NumPy materializer. Here those operations become the language the agent writes. The following sections explain the small set of objects involved, what the agent sees through `orient`, and three existing retrieval programs expressed in Algebra, SQL, and PyTerrier.

2 Preliminaries

Let `E` contain one embedding per record and `q` be a query vector. Then `E @ q` assigns a similarity score to every candidate. With normalized rows and a unit query, these scores equal cosine similarity. An identity stays attached to each score so that results can be joined to the underlying records.

The arithmetic in this paper uses queries in the same embedding space. `+` adds their scores, `-` subtracts them, and scalar `*` scales a contribution. For example, `(E @ q1) - 0.5 * (E @ q2)` favors the first direction while suppressing the second. Selection happens after composition; subtracting two already-truncated top-ten lists would be a different operation.

A mask `m` is a set of eligible identities. `m ▷ S` removes other records from a scored relation `S`. A weight relation `w` supplies a scalar for each eligible identity, and `w ⊙ S` multiplies its score. Masks remove records; a zero weight leaves a record present with score zero. Finally, $\tau_{10}(S)$ selects up to ten results, ordered by score descending and identity ascending. The ASCII spelling is `top(10, S)`.

3 Architecture

The architecture follows PEM's division of work. The current materializer uses SQLite to resolve operands and NumPy to evaluate vector arithmetic, then returns a temporary relation that SQL can hydrate, join, or group. The existing parser and planner determine the operations from the agent's expression.

```
agent expression
    → parse and plan
    → materializer executes the retrieval program
    → scored SQL relation
```

The agent submits a program through `algebra('<expression>')`. It can also place that constructor inside a larger SQL query. Changing a subtraction coefficient or adding a mask changes the retrieval program without requiring a new endpoint.

PEM's `similar:` and `suppress:` tokens requested numerical operations through SQL. Algebra expresses their composition directly. Before scoring, the planner can collapse:

```
(E @ q1) - 0.5 * (E @ q2) = E @ (q1 - 0.5 * q2)
```

This identity holds in real arithmetic. The implementation preserves the composed vector's magnitude and performs one matrix multiplication; float32 evaluation orders can differ slightly.

The SQL listings below are executable equivalents, not compiler output. They use sqlite-vec [10] to calculate cosine similarity directly from stored vectors, and run over the fixture's `embeddings(docno, vector)` and the views `m(id)` and `w(id, weight)` defined at the end of the paper.

4 Orient: what the agent sees

Before writing a query, the agent calls:

```
algebra('orient')
```

The response identifies available operands, their underlying relations, and whether they can be used. These are the six operand rows returned for the Vaswani proof fixture:

section	token	domain	status
relation	`R`	`documents(docno, text)`	`ok`
matrix	`E`	`embeddings(docno, vector)`	`ok`
query	`q1`	`query_vectors(qid, vector)`	`ok`
query	`q2`	`embeddings(docno, vector)`	`ok`
mask	`m`	`pyterrier_bm25_results(docno, qid)`	`ok`
weight	`w`	`pyterrier_bm25_results(docno, score, qid)`	`ok`

The same response advertises scoring, restriction, weighting, fusion, and selection. The agent sees both usable symbols and their underlying schema. Whoever integrates the corpus declares the bindings between symbols and relations; this fixture declares them in the proof runner. The agent composes within that namespace.

The fixture contains 11,429 documents and deterministic 128-dimensional vectors. `q1` is the stored query vector for "What are chemical reactions?"; `q2` is a fixed vector from document `2008`. The mask contains the 52 candidates in the recorded PyTerrier BM25 result for that query. Each weight is its BM25 score divided by the maximum score in that result. The vectors make the computation reproducible; they are not a learned-embedding retrieval evaluation.

5 Queries the agent can compose

The following examples retain the operations and inputs of three existing proof cases. For the PyTerrier forms, `D` supplies every document, `M` supplies the recorded BM25 candidates, `A` and `B` score with the two query vectors, and `W` applies the declared weights. `T` explicitly orders by score and document identity before cutoff. Their native PyTerrier definitions appear at the end of the paper.

5.1 Subtract one similarity search from another

The agent can use a second query or an example document to specify an unwanted direction. Here document `2008` fixes the subtraction operand reproducibly; it is not assigned an irrelevance judgment.

Algebraic expression

```
τ10((E @ q1) - 0.5 * (E @ q2))
```

Equivalent SQL

```
SELECT docno AS id,
       (1 - vec_distance_cosine(vector, :q1))
       - 0.5 * (1 - vec_distance_cosine(vector, :q2)) AS score
FROM embeddings
ORDER BY score DESC, id ASC
LIMIT 10;
```

PyTerrier

```
(D >> (A + (-0.5) * B) >> T) % 10
```

PyTerrier adds the first scorer to a negatively weighted second scorer. The SQL alternative calculates two primitive cosine similarities per row; Algebra combines the vectors and scores once. All select the same ten identities, with document `9448` first. PyTerrier returns the same order, with a maximum score difference of `2.24e-8`. Documents `7429` and `9217` tie exactly in Algebra and PyTerrier, where identity ordering places `7429` first. The SQL path separates them by approximately `5.96e-8` and places `9217` first; its maximum returned-score difference is also approximately `5.96e-8`.

Replacing subtraction with addition expresses a blend. This is the practical meaning of composing searches with `+`, `-`, and `*`: the agent writes the scoring expression it needs.

5.2 Rerank a candidate pool

The agent restricts vector scoring to an existing first-pass result. Here the mask is the 52-document BM25 candidate pool exposed by `orient`.

Algebraic expression

```
τ10(m ▷ (E @ q1))
```

Equivalent SQL

```
SELECT e.docno AS id, 1 - vec_distance_cosine(e.vector, :q1) AS score
FROM embeddings AS e JOIN m ON m.id = e.docno
ORDER BY score DESC, id ASC
LIMIT 10;
```

PyTerrier

```
(M >> A >> T) % 10
```

This is the familiar candidate-generation → reranking → cutoff pipeline. The Algebra planner applies the mask before its vector scoring pass. The SQL join expresses the same allowed identities, and PyTerrier passes those candidates to the scorer. All return the same ordered identities, with document `2417` first. PyTerrier scores match Algebra exactly; the SQL scores have a maximum observed difference of approximately `2.98e-8`.

5.3 Weight the masked results

The agent adds a weight to the previous program. This fixture uses normalized BM25 scores to demonstrate multiplication, without proposing an optimal hybrid ranking formula.

Algebraic expression

```
τ₁₀(w ⊙ (m ▷ (E @ q1)))
```

Equivalent SQL

```
SELECT e.docno AS id,
       w.weight * (1 - vec_distance_cosine(e.vector, :q1)) AS score
FROM embeddings AS e
JOIN m ON m.id = e.docno JOIN w ON w.id = e.docno
ORDER BY score DESC, id ASC
LIMIT 10;
```

PyTerrier

```
(M >> A >> W >> T) % 10
```

All 52 eligible identities have weights, and restriction precedes weighting in the expression. All forms return the same ordered identities; PyTerrier scores match Algebra exactly, and SQL differs by at most `2.61e-8`. Weighting moves document `9374` to first place, using the same candidate pool as the previous example.

The examples change a few terms in one expression: subtract a search, introduce a mask, then apply weights. The result of each remains a scored relation that the agent can inspect and use in another query.

6 Explanation and evidence

The composition works because the intermediate scores remain available with their identities. Arithmetic changes their values, a mask changes which identities participate, and top-k selects the final result. The same objects pass between these operations.

The PyTerrier 1.1.2 programs use independent FAISS 1.15.0 `IndexFlatIP` scores. The SQL alternatives compute scores directly with sqlite-vec 0.1.9 over the stored vectors. Every selected document set matches Algebra and all score differences are below `1e-6`; §5.1 reports the ordering difference. These are computation checks, not timing comparisons. The candidate pool is the recorded upstream BM25 result, so the checks do not compare BM25 implementations.

Score tolerance does not imply rank agreement. Algebra and PyTerrier tie `7429` and `9217` at `0.24868088960647583`, a reference gap of zero; SQL favors `9217` by approximately `5.96e-8`. Identity ordering resolves only computed ties. If each alternative score differs from its reference by at most ε, a reference gap greater than 2ε preserves their order.

The uncollapsed §5.2 comparison already shows SQL deviations of approximately `2.98e-8`. A two-pass NumPy calculation of §5.1 preserves the tie, so this swap does not require collapse. Stored float32 rows are approximately unit length; sqlite-vec 0.1.9 [10] recomputes norms using float32 accumulation, then rounds cosine distance before SQL subtracts it from one. Reproducing those steps reproduces the pair's split: a deterministic rounding effect.

The existing six-case proof also covers score thresholds. Portable regressions check restriction versus zero-weighting on negative scores and identity tie ordering, collapse, and threshold membership. A feedback regression executes score → top-1 → centroid → rescore: moving the mask changes the result order, so that stage boundary must be preserved.

PyTerrier already provides an algebra of retrieval pipelines [3]. Vector scoring and feedback [1,2], relational top-k [6,7], Vespa ranking expressions [8], and semantic relational operators such as LOTUS [9] supply established context. The contribution here is the agent-facing mathematical surface over live operands, together with executable examples of its composition. The experiments do not measure retrieval-quality gains, ANN recall, comparative agent query-writing accuracy, or universal compiler correctness.

7 Conclusion

An agent can write retrieval as arithmetic over similarity scores, restrict it with a mask, and change the program at query time. The existing runtime executes these short expressions and returns relations the agent can continue to compose. On the public fixture, three implementations select the same document sets with score differences below `1e-6`, yet one tied pair orders differently. Numerical agreement must be checked separately from rank agreement.

References

1. G. Salton, A. Wong, and C. S. Yang. "A Vector Space Model for Automatic Indexing." *Communications of the ACM*, 1975.
2. J. J. Rocchio. "Relevance Feedback in Information Retrieval." *The SMART Retrieval System*, 1971.
3. Craig Macdonald and Nicola Tonellotto. "Declarative Experimentation in Information Retrieval using PyTerrier." *ICTIR*, 2020.
4. Sean MacAvaney et al. "Simplified Data Wrangling with ir_datasets." *SIGIR*, 2021.
5. Damian Delmas. "flexvec: SQL Vector Retrieval with Programmatic Embedding Modulation." arXiv:2603.22587, 2026. DOI: 10.48550/arXiv.2603.22587.
6. Chengkai Li, Kevin C.-C. Chang, Ihab F. Ilyas, and Sumin Song. "RankSQL: Query Algebra and Optimization for Relational Top-k Queries." *SIGMOD*, 2005. DOI: 10.1145/1066157.1066173.
7. Ronald Fagin, Amnon Lotem, and Moni Naor. "Optimal Aggregation Algorithms for Middleware." *PODS*, 2001. DOI: 10.1145/375551.375567.
8. Vespa. "YQL Query Language," "Ranking Expressions," and "Phased Ranking." Vespa Documentation, accessed 2026.
9. Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. "Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS." *PVLDB*, 2025.
10. Alex Garcia. "sqlite-vec: API Reference." https://alexgarcia.xyz/sqlite-vec/api-reference.html. Version 0.1.9 used in the SQL comparisons; cosine implementation.

Example bindings

These are the bindings used to check the listings. `ids` contains all document identities; `score1` and `score2` map them to the full independent FAISS primitive scores. `bm25` contains the recorded first-pass result. Each pipeline is executed with `.transform(topics)` for the single fixture query.

```
weight = dict(zip(bm25.docno, bm25.score / bm25.score.max()))
D = pt.Transformer.from_df(pd.DataFrame({"qid": "1", "docno": ids}))
M = pt.Transformer.from_df(bm25)
A = pt.apply.doc_score(lambda r: float(score1[r["docno"]]))
B = pt.apply.doc_score(lambda r: float(score2[r["docno"]]))
W = pt.apply.doc_score(lambda r: r["score"] * weight[r["docno"]])

def score_id_order(frame):
    out = frame.sort_values(
        ["score", "docno"], ascending=[False, True], kind="mergesort"
    ).reset_index(drop=True)
    out["rank"] = np.arange(len(out))
    return out

T = pt.apply.generic(score_id_order)
```

The explicit ordering stage supplies the comparison's tie policy. PyTerrier's default tied ranks follow input order. Code, fixture instructions [4], and dependencies are available at `https://github.com/algebraicretrieval/algebraicretrieval/tree/main/proofs/pyterrier`. Its `examples.py` runs all three comparisons, the two-pass scoring check, and the scalar rounding reproduction:

```
python proofs/pyterrier/examples.py
```

The native SQL alternatives bind `:q1` to the stored vector for query `1` and `:q2` to document `2008`'s embedding, and use these views. SQL's window maximum and PyTerrier's `bm25.score.max()` both cover the same 52 rows with `qid='1'`; the maximum is `22.076426496954774`, and the normalized weights agree exactly.

```
CREATE TEMP VIEW m AS
SELECT docno AS id FROM pyterrier_bm25_results WHERE qid='1';
CREATE TEMP VIEW w AS
SELECT docno AS id, score / max(score) OVER () AS weight
FROM pyterrier_bm25_results WHERE qid='1';
```